

1 操作系统概论

1.1 操作系统与计算机系统

计算机系统由硬件和软件两个部分组成,硬件层提供基本的可计算性资源,软件由程序、数据、文档组成,可以分为系统软件(操作系统)、支撑软件、应用软件。

操作系统与支撑软件、应用软件的区别:

操作系统有权分配资源而另外 2 个只能通过操作系统使用资源,是控制与被控制的关系。操作系统直接作用于硬件之上,隔离其他上层软件,并为之提供接口和服务。因此,操作系统是软件系统的核心,是各种软件的基础运行平台。

1.2 操作系统资源管理技术

复用、虚拟、抽象

其中,复用和虚拟解决物理资源数量不足的问题,抽象解决系统的复杂性,实现资源的易用性。

①复用

多道程序设计是现代操作系统采用的基本技术,系统中有多个进程竞争使用资源。资源复用就是操作系统让众多进程共享物理资源的这种共享。物理资源的复用共享有 2 种基本方法:空分复用共享和时分复用共享,时分复用又分为时分独占式(磁带机、扫描仪)、时分共享式。

②虚拟

可以把一个物理资源转变成多个逻辑上的对应物,也可以把多个物理资源变成单个逻辑上的对应物。

虚拟技术用于外部设备就是外部设备同时联机操作(SPOOLing)技术,让物理上的一台独占设备转化成逻辑上的多台虚拟独占设备,每个进程都有自己的虚拟打印机,多个进程可以并行打印。

虚拟技术用于文件系统就是虚拟文件系统(VFS),使操作系统可以在 VFS 控制下同时支持多种具体文件系统,让多个物理资源变成单个逻辑上的对应物(VFS)。

③抽象

操作系统对物理资源进行抽象,通过创建软件来屏蔽硬件资源的物理特性和实现细节,在硬件接口的基础上,把按特定方式操作的底层指令序列和数据结构打包成函数,作为单一的高级操作被调用,用户只要通过软件接口(系统调用,例如应用程序可以使用 fork()用于创建一个新进程,wait(),exec()等,也有文件系统调用 read(),write(),open()等)就可以操作和使用物理资源。

抽象技术也可用于定义和构造多层软件抽象,每层软件都隐蔽下一层的实现细节,从而形成多级资源抽象。数据 I/O 就依赖于多层抽象。

操作系统中的基础抽象有进程抽象、虚存抽象、文件抽象。

另外,操作系统虚拟机是在裸机上配置操作系统,由操作系统对物理计算机实施仿真而获得,虚拟机的每种虚拟资源都是物理资源通过复用、虚拟或抽象得到的产物。

1.3 操作系统的定义

操作系统是管理系统资源、控制程序执行、改善人机界面、提供各种服务,并合理组织计算机工作流程和为用户方便有效地使用计算机提供良好运行环境的一种系统软件。

1.4 操作系统的概述

操作系统是最靠近硬件的一层软件,它把硬件裸机改造成完善的虚拟机,是及其功能得到扩展,运行环境得到改善,系统效率得到提高,安全性能得到保证。隔离其他上层软件,并为之提供接口和服务,使上层软件可以获得比硬件更多更强的功能上的支持。操作系统是

软件系统的核心，它与硬件一起构成了各种软件的基础运行平台。



在层次化的计算机系统模型中，操作系统是（ ）

- A 覆盖在硬件裸机上的第一层（大型复杂）软件
- B 面向计算机系统中的（所有程序的）共性问题
- C 编译程序
- D 电子商务

1.5 操作系统在计算机系统中起 4 个方面的作用

- ①服务用户观点——操作系统作为用户接口和公共服务程序
- ②进程交互观点——操作系统作为进程执行的控制者和协调者
- ③系统实现观点——操作系统作为扩展机或虚拟机
- ④资源管理观点——操作系统作为资源的管理者和控制者

操作系统既是管理员，又是服务员。

1.6 操作系统的功能

从资源管理的观点来看，操作系统有处理机管理、存储器管理、设备管理、文件管理、联网与通信管理或用户接口这 5 项功能。

1.7 操作系统的特性

并发性、共享性、异步性

并发、共享、虚拟、异步性

并发和共享是操作系统 2 个最基本的特征，二者互为存在条件。

①并发

并发性是指两个或两个以上的活动或事件在同一时间间隔内发生。

操作系统是一个并发系统，有多道程序同时运行，这些程序就是并发程序，这样的系统就是并发系统。采用并发技术的系统又称为多任务处理系统，并发技术实际上是物理 CPU 在若干道程序之间的多路复用，从而实现运行程序之间的并发以及 CPU 与设备、设备与设备之间的并行。并发性的实质是对有限的物理资源强行复用，供多用户共享以提高效率。

然而，在单 CPU 系统中，多个程序并发执行是宏观上的概念，微观上仍是顺序执行的；在多 CPU 系统中，并发性既体现在宏观上，又体现在微观上，这就叫并行。

并行性是指两个或两个以上的活动事件在同一时刻发生。

并行活动一定是并发的，并发活动未必是并行的。并行性是并发性的特例，并发性是并行性的扩展。

总结一下：

并发指的是计算机系统中同时存在多道运行的程序，宏观上表现为多道程序同时执行，但是微观上任何时刻只有一道程序执行，多道程序在 CPU 上轮流、交替地执行。

并行才真正指的是多道程序在同一时刻执行，但是需要多个硬件支持。

程序的并发执行能有效改善系统的资源利用率，并发是操作系统最重要的特征，其他特征均以并发为前提。

②共享

计算机系统资源可以被多个并发执行的程序共同使用而不是被某个程序独占。

资源共享的方式有透明资源共享、独占资源共享 2 种（也可以说同时访问式、互斥共享式）。

③虚拟

把一个物理实体映射为多个与之对应的逻辑实体。

④异步性（随机性/不确定性）

在多道程序环境中允许多个程序并发执行，并发活动会导致随机事件的发生，一些情况不可预知，且由于资源有限而程序众多，每个程序的执行并不是连贯的，而是“走走停停”。

1.8 多道程序设计

多道程序设计是指允许多个作业（程序）同时进入计算机系统的内存并启动交替计算的方法。内存中多个相互独立的程序都处于开始和结束之间，从宏观上看是并行的，多道程序都开始运行但并未执行结束；从微观上看是串行的，多个作业轮流占用 CPU 交替执行。

CPU 利用率：

假定程序平均等待 I/O 操作的时间占其运行时间的比例是 p ，当程序的道数或度数为 n 时，所有程序都等待 I/O 操作的概率为 p^n ，此时 CPU 的利用率就是 $1-p^n$ 。

吞吐量：

单位时间完成的作业数。

优点：

- ①提高 CPU、内存、设备的利用率
- ②提高系统的吞吐率，使单位时间内完成的作业数量增加
- ③充分发挥系统的并行性，使设备与设备之间、CPU 与设备之间都可以并行工作

缺点：

延长了作业的周转时间

多道程序设计系统与多重处理系统的区别：

多重处理系统是指配置多个物理处理器，能真正同时执行多道程序的计算机系统。当然，要有效地使用多重处理系统，则必须采用多道程序设计技术；而多道程序设计不一定要求多重处理系统的支持。



计算机系统中，引入多道程序设计后，

- A** 增大了单位时间内的解题数量（吞吐率）
- B** （可能会）延长（某些）程序的等待时间
- C** 提高了系统的资源利用率
- D** 可以充分发挥系统的并行性

1.9 操作系统的发展与分类

一、无操作系统的计算机系统

- (1) **人工操作方式**
- (2) **脱机输入输出方式**

二、批处理系统

- 1.2.2 单道批处理系统**
- 1.2.3 多道批处理系统**

三、分时系统

四、实时系统

五、微机操作系统

六、多处理机操作系统

七、网络操作系统

八、分布式操作系统

九、嵌入式操作系统

批处理操作系统->分时操作系统->实时操作系统->.....

批处理操作系统服务于一系列称作“批”的作业，系统资源利用率高、作业吞吐量大，

但是作业周转时间长、不具备交互式计算能力，不利于程序的开发和调试。适用于已经调试好的大型作业。单道批处理系统中内存仅存放一道作业，资源利用率和吞吐量低下。多道批处理系统中的输入井和输出井分别是磁盘上的两个区域，为了协调 I/O 设备与处理机之间的速度差异，为作业调度提供了条件，体现了“多道”和“成批”的两个特征。

分时操作系统采用**时间片轮转**的方法，把处理器的时间划分成时间片轮流分配给各个联机终端，如果时间片用完则产生时钟中断，控制权交由操作系统重新调度；如果源程序尚未完成则挂起等待重新分得时间片。具备同时性、独立性、及时性、交互性的特点。适用于正在调试的小型作业。

它们都是基于多道程序设计技术的。

实时操作系统能及时响应外部事件的请求，并控制所有实时任务协调一致地运行。具有响应及时、可靠性高的特点。

如果某操作系统兼具上面 3 种功能中的 2 种及以上，那么它就是通用操作系统。

1.10 系统调用

操作系统通过**程序接口（系统调用）**和操作接口（系统程序）来将其功能和服务提供给用户。

应用程序只有通过系统调用才能请求系统服务并使用系统资源。

由于系统调用而引起处理器中断的机器指令称为访管指令、自陷指令或中断指令。其中访管指令是非特权指令。

作用：

- ①内核可以基于权限和规则对资源访问进行裁决，保证系统的安全性；
- ②系统调用对资源进行抽象，提供一致性接口，避免用户使用资源时发生错误，提高编程效率。

分类：

进程管理、文件管理、设备管理、存储管理、进程通信、信息维护。

实现：

操作系统实现系统调用功能的机制是陷阱，亦称异常处理机制。

- ①编写系统调用服务例程
- ②设计系统调用入口地址表，每个入口地址都指向一个系统调用服务例程
- ③陷阱处理机制，开辟现场保护区以保存发生系统调用时应用程序的处理器现场

应用程序执行系统调用，产生中断转向内核态，进入陷阱处理程序，按功能号查询入口地址表，转至对应服务例程执行，完成后退出中断，返回应用程序断点处继续执行。

API、库函数、系统调用的区别：

- ①API 是编程规范定义的接口集合，库函数是 API 的一种具体实现形式；
- ②部分库函数通过封装系统调用实现功能，而另一部分库函数完全在用户态执行；
- ③应用程序通过调用库函数间接地进行系统调用或者直接调用系统调用接口，操作系统通过系统调用向用户程序提供服务。

系统调用与函数调用的区别：

- ①调用形式和实现方式不同

函数调用转向的地址是固定不变的，系统调用中不包含内核服务例程的入口地址，仅按功能号调用。

函数调用是在用户态执行的并访问用户栈，系统调用要通过陷阱机制从用户态转向内核态，服务例程在内核态执行并访问核心栈。

- ②被调用代码的位置不同

函数调用是静态调用，编译链接后形成用户级程序，它发生变化时必须重新编译链接。

系统调用是动态调用,调用的服务例程位于操作系统中,它发生变化时与调用程序无关,而且长度短,减少占用的内存空间。

③提供方式不同

函数调用由编程语言提供,不同语言提供的就不同。

系统调用由操作系统提供,一旦操作系统设计好则固定不变。

下面这道题目选错误的:

 Linux中c程序中有printf语句,运行该语句输出时,以下说法正确的是()

- ☐ A 需要系统调用才能有输出
- ☐ B 程序运行会进入内核态
- ☒ C 该语句能直接将输出内容输出
- ☐ D printf是C语言的库函数

1.11 操作系统的结构分类

结构是影响操作系统正确性和性能的重要因素。

单体式结构、层次式结构、虚拟机结构、微内核结构

1.12 操作系统内核

通常把组成操作系统的基本单位称作构件。

内核是操作系统对裸机的第一次改造, 内核与裸机组成第一层虚拟机,进程在虚拟机上运行。

虚拟机的特性:

- ①虚拟机没有中断
- ②虚拟机为每个进程提供一台虚处理器,制造了一种每个进程在各自处理器上顺序推进的假象,实现多进程并发执行
- ③虚拟机为进程提供强大的指令系统,它们能够使用机器非特权指令和系统调用所组成的新的指令系统。

内核是一组程序模块,作为可信软件来支持进程并发执行的基本功能和基本操作,通常驻留在内核空间,运行于内核态,具有直接访问硬件设备和所有内存空间的权限,是仅有的能够执行特权指令的程序。

内核的大小与功能有很大差别,例如微内核(功能少)、单内核(多功能以提高系统效率)。

属性:

- ①内核是由中断驱动的
- ②内核是不可抢占的
- ③内核可以在屏蔽中断状态下执行
- ④内核可以使用特权指令

功能:

中断处理、时钟管理、短程调度、原语管理

原语和系统调用的区别?

原语和系统调用都使用访管指令实现,具有相同的调用形式,但是原语由内核实现,系统调用由系统进程实现;原语不允许被中断,系统调用允许被中断;原语供系统进程使

用，但是原语绝对不会调用系统进程。

补充：

通道是计算机，但是计算器不是。

机制与策略分离原则是让程序中的独立部分分别来实现机制与策略，这有助于使内核保持短小且良好的结构，这种软件适应性好、易于开发。大致的思路是，通过将机制植入操作系统而将策略留给上层软件（如应用进程）。

1.13 操作系统运行模型

① “操作系统服务例程嵌入应用进程中”的模型

应用程序通过 API 请求操作系统提供服务，API 再调用 C 函数库中对应的封装函数，通过陷阱机制进入内核，执行相应的服务例程，这样就可以让服务例程成为应用进程的子程序。

② “操作系统服务例程作为独立进程运行”的模型

操作系统调用的服务例程功能由一组用户态的系统进程来完成。

2 处理器管理

2.1 特权指令与非特权指令

机器指令集合称指令系统。

特权指令是指仅在内核态下才能使用的指令，非特权指令是指在用户态和核心态下都能工作的指令。操作系统程序能够执行全部机器指令，应用程序只能使用非特权指令。

2.2 内核态与用户态

根据执行程序对资源和机器指令的使用权限将处理器设置成不同的状态，至少需划分为内核态、用户态两种。

处理器建立硬件标志位，称为处理器状态位，是程序状态字中的一位，用来区分可信软件和非可信软件。当处理器处于内核态时，可以认为其正在运行可信软件；当处理器处于用户态时，可以认为其正在运行非可信软件。

处理器状态及转换：

有三种情况会造成处理器从用户态向内核态转换：

①程序请求操作系统服务，执行系统调用

②程序运行时产生中断事件

③程序运行时产生异常事件

内核态向用户态转换：

①加载程序状态字的特权指令 iret

可以说，中断和异常是用户态到内核态转换的仅有途径：当系统中产生中断或异常，处理器将响应并交换程序状态字，此时会导致处理器从用户态转向内核态。中断事件或异常处理程序的程序状态字中的处理器状态位一定是“内核态”。



将CPU区分为多个状态，其一为核心态（内核态），主要是（ ）

- ☐ A 不管CPU复杂与简单，没有状态区分也可以
- ☒ B 便于实现对CPU等资源的使用管理
- ☐ C 在该状态下不能运行特权指令

2.3 用户栈与核心栈

用户栈是用户进程空间中的一块区域，核心栈是操作系统空间中的一块区域。

区分：

①存在位置不同：

用户栈在用户进程空间中开辟，而核心栈在内存中单独开辟。

②作用不同：

用户栈保存应用程序的子程序间相互调用的参数、返回值、返回点及子程序的局部变量等。

核心栈在操作系统空间区域，一是保存操作系统程序（函数）间相互调用的参数，返回值以及程序的局部变量；二是保存中断现场信息。



如果CPU执行操作系统内核程序时，需要有能执行全部指令的权限，此时CPU处于（ ）状态

- A 目态
- B 用户态
- C 内核态
- D 权限最低级别

2.4 程序状态字（PSW-Program Status Word）

操作系统将程序运行时的一组动态信息汇集在一起，形成程序状态字，存放在处理器的一组特殊寄存器中，主要是为了实现程序状态的保护与恢复，对程序运行进行控制与管理，用来指示程序的运行状态、控制指令执行顺序、保留与程序运行有关的各种信息。

每个程序都有一个与其执行相关的 PSW，每个处理器都设置一个 PSW 寄存器，程序占有处理器执行，其 PSW 占有 PSW 寄存器。

2.5 中断的定义

中断指在程序执行过程中遇到急需处理的事件时，暂时终止现执行程序在 CPU 上的运行，转而执行相应的事件处理程序，待处理完成后返回断点或调度其他程序执行的过程。

可以说，操作系统是由中断驱动的。

2.6 中断源的分类

外中断、内中断

由硬件发出的中断是硬中断，按硬中断事件的来源和实现手段可将中断划分为外中断和内中断。

外中断又称中断或异步中断，可再分为可屏蔽中断和不可屏蔽中断。

内中断又称异常或同步中断，可再分为访管中断（由系统调用引起）、硬件故障中断、程序性异常。

中断信号源分为中断和异常两类。

2.7 中断与异常的区别

中断是由与现执行程序无关的中断信号触发的，与 CPU 是异步的，只有在两条机器指令之间才可以响应中断，CPU 对中断的响应完全是被动的。中断的发生与 CPU 当前状态无关，既可以发生在用户态又可以发生在内核态。一般来说，中断处理程序提供的服务不是当前进程需要的，它的处理过程是不能阻塞的。中断允许发生嵌套。中断处理过程中不会被异常打断。

异常（例如出错 fault、陷入 trap、终止 abort、编程异常）是由现行指令引起的，与 CPU 是同步的，允许在一条指令的执行期间响应异常。大部分异常发生在用户态，唯一在

内核态发生的异常是缺页异常。异常处理程序提供的服务是当前进程需要的，它的处理过程是可以阻塞的。异常大多为一重。异常处理过程中可能产生中断。

2.8 中断与异常的响应和服务

产生中断->关中断->保存断点->引出中断服务程序->保存现场和屏蔽字->开中断->执行中断服务程序->关中断->恢复现场和屏蔽字->开中断->中断返回。

①发现中断源

在中断未被屏蔽的前提下，硬件发现中断/异常事件，并由 CPU 响应中断/异常请求。

当发现多个中断源时，将根据预定的中断优先级先后响应中断请求。

②保护现场

暂停当前程序运行，硬件将中断点的现场 PSW 保存至核心栈。

③转向中断/异常事件处理程序执行

处理器状态由用户态切换到内核态，查中断向量表转入相应处理程序。

④恢复现场

当中断处理结束后，恢复原运行程序的 PSW，重新返回中断点执行后续指令。但是异常处理结束后，返回点因异常类型而异。

2.9 中断优先级

如果系统正在执行某优先级的中断服务例程，那么只有更高优先级的中断请求才能中断该服务例程。

中断屏蔽：

禁止 CPU 响应中断，或者禁止中断产生。

多重中断事件：

如果是同一优先级的不同中断，就由同一个中断处理程序从左至右地逐个处理。

如果是不同优先级的中断，可分不同情况进行串行处理、嵌套处理、即时处理。

■ 2023年全国统考题 24

在操作系统内核中，中断向量表适合采用的数据结构是
()

A、数组

B、队列

C、单向链表

D、双向链表



- 1、下列选项中，在用户态执行的是_____。
A. **命令解释程序** B. 缺页处理程序
C. 进程调度程序 D. 时钟中断处理程序
- 2、下列选项中，不可能在用户态发生的事件是_____。
A. 系统调用 B. 外部中断 C. **进程切换** D. 缺页
- 3、响应外部中断的过程中，中断隐指令完成的操作，除保护断点外，还包括_____。
I. **关中断**
II. 保存通用寄存器的内容
III. **形成中断服务程序入口地址并送 PC**
A. 仅 I、II B. **仅 I、III**
C. 仅 II、III D. I、II、III
- 4、中断处理和子程序调用都需要压栈以保护现场，中断处理一定会保存而子程序调用不需要保存其内容的是_____。
A. 程序计数器 B. **程序状态字寄存器**
C. 通用数据寄存器 D. 通用地址寄存器

2.10 中断请求队列中，中断处理程序和中断服务例程的区别

中断处理程序是对应某一中断向量的总处理入口，负责触发该中断线关联设备的中断服务程序；中断服务例程是具体设备的中断处理逻辑。

2.11 进程的定义

进程是进程实体的运行过程，是系统进行资源分配和调度的一个独立单位。

2.12 引入进程的两个目的

- ①刻画程序的并发性
- ②解决资源的共享性

2.13 进程的特性（进程与程序的区别）

动态性、共享性、独立性、制约性、并发性

①动态性

进程是程序在处理机上的一次执行过程，它是动态的概念，它是有生命周期的；程序是指令的有序集合，它是静态的概念，在系统中作为一种永久存在的资源。

②共享性

同一程序同时运行于不同的数据集合上会构成不同的进程，多个进程可以运行相同的程序，进程和程序不是一一对应的。

③独立性

每个进程是操作系统中一个独立的实体，有自己的虚存空间、程序计数器和内部状态。

④制约性

进程因共享资源或协同工作而产生相互制约关系，必须协调它们的执行次序或者相对执行速度。

⑤并发性

多个进程的执行在时间上可以重叠，在单处理系统中并发执行，在多处理系统中并行执行，进程的执行是可以被打断的。

进程与程序的主要区别

	程序	进程
概念	静态	动态
所在存储器	外存	内存
存在时间	永久	有生命期
组成	有序指令	程序段,数据段,PCB
对应关系	一个程序可对应多个进程 一个进程可对应某个程序段	

2.14 进程的状态及转换

①三态模型

运行态、等待态、就绪态

1、进程的5种状态 (三种基本状态)

进程可能由于等待I/O操作、竞争资源、以及相互协作等原因产生了“走走停停”的动态性。因此，进程在生存期内至少具有三种基本状态：



②七态模型

2.15 导致进程创建的原因

新的批处理作业、交互登录、操作系统提供服务、现有进程派生.....

2.16 导致进程终止（撤销）的原因

进程正常完成、无可利用内存、父进程终止、父进程请求、时间超出、地址越界、保护权限错、数据溢出、I/O 失败、进程在用户态执行了特权指令、无效指令、数据误用、系统操作员或操作系统干涉.....

2.17 引起挂起状态的原因

对换（操作系统要求释放更多的主存空间来调入并执行就绪态进程）、操作系统检查、交互式用户要求、父进程请求、定时.....

2.18 进程映像

进程由四要素组成：

进程控制块、进程程序块、进程核心栈、进程数据块

2.19 进程上下文

进程物理实体和支持进程运行的环境的合称。

当系统调度新进程占有处理器时，新老进程随之发生上下文切换。进程的运行被认为是在上下文中执行。

三部分组成：

用户级上下文、寄存器上下文、系统级上下文

2.20 进程控制块

PCB 是进程的唯一标识，操作系统用于记录和刻画进程状态及其有关信息的数据结构，也是操作系统掌握进程的唯一资料结构，包括了进程执行时的情况以及进程让出处理器后所处的状态、断点等信息。创建进程时必须为其创建进程控制块，撤销进程时回收。包含三类信息：标识信息、现场信息、控制信息；两种组织方式：链接方式、索引方式。

2.21 进程队列及其管理

进程的特征主要由 PCB 刻画，为了对进程进行管理和调度，将进程的 PCB 通过某种方法组织起来。一般地，把处于同一状态的所有进程的 PCB 链接在一起的数据结构称为进程队列。

同一状态进程的 PCB 既可按先来先到的原则排成队列;也可按优先数或其它原则排成队列。

两种组织方式:

链接方式、索引方式

2.22 进程上下文切换

进程切换是让处于运行态的进程中断运行，让出处理器，这时要做一次进程上下文切换、即保存老进程的上下文且装入新进程的上下文，以便新进程运行。

中断和异常是激活操作系统仅有的两种手段，它暂停现行进程的执行，把处理器转至内核态，内核获得处理器控制权之后就可以进行上下文切换。进程切换必定发生在内核态。

步骤:

- ①保存被中断进程的处理器现场信息
- ②修改被中断进程 PCB 的有关信息，将其 PCB 加入相关队列
- ③选择占用处理器运行的另一个进程，修改其 PCB 信息
- ④设置被选中进程的地址空间并恢复存储管理信息
- ⑤根据被选中进程的上下文信息恢复处理器现场

2.23 处理器模式切换

当发生中断或系统调用时暂停正在运行的进程，把处理器状态由用户态转向内核态，执行操作系统服务例程。此时进程仍然正在自己的上下文中执行，只有处理器的状态发生变化，内核在被中断进程的上下文中进行处理。

步骤:

- ①保存被中断进程的处理器现场信息
- ②根据中断号置程序计数器
- ③把用户状态切换到内核状态，以便执行中断处理程序

CPU 模式切换不一定会引起进程上下文切换，但是进程调度一定会引起 CPU 模式切换（用户态→内核态→用户态）

2.24 进程控制和管理

操作系统通过原语来对进程进行控制和管理，进程的创建与撤销，阻塞（block()，将进程由运行态转为阻塞态）与唤醒（wakeup()将进程由阻塞态变为就绪态），挂起（suspend()）与激活（active()）都由它来完成。

原语(Primitive)是在管态下执行、完成系统特定功能的过程。原语的特点是执行过程中不允许被中断，是一个不可分割的基本单位，原语的执行是顺序的而不可能是并发的。

2.25 线程

线程是处理器调度的基本单位，是进程内一个相对独立的执行流或控制流，是处理机分配的实体。

线程控制块叫 TCB。

线程之间有两种常见的交互，一种是访问共享变量，要使临界区有原子性，访问临界区是原子操作；线程之间可以相互阻塞和唤醒。

好处:

- ①并发执行在线程实体上，即处理机（调度）分配发生在线程之间，改善系统时空开销。
- ②系统创建或终止一个线程的开销要比创建或终止一个进程的开销少得多。

③线程之间通信效率要高于进程之间的通信效率。进程间通信要内核介入，而同一进程多个线程共享同一地址空间，所以通信无须内核介入。

分类：

线程的实现有用户级线程、内核级线程、混合式线程这三种。

①用户级线程

因为用户级线程的上下文切换与内核无关，用户级线程的调度算法只进行线程上下文切换，而不涉及处理机的状态。所以当一进程由于 I/O 中断等原因使该进程阻塞，而属于该进程的执行中线程仍处于执行状态。

进程内线程切换避免了用户态->内核态->用户态的切换，减少系统开销。

一个用户级线程的阻塞将引起整个进程阻塞。

②内核级线程

同一进程内的线程切换需要用户态->内核态->用户态的 CPU 模式切换，系统开销大。

倘若进程中的一个线程被阻塞，内核可以调度其他线程占用处理器运行。

注意，同一进程内的线程切换不会引起进程切换，但是不同进程中的线程切换会引起进程切换。

2.26 进程与线程的对比

①它们都具有并发性，都能够提高资源利用率和系统吞吐量，但是线程的并发粒度更细，并发执行的程度更高。

②进程是操作系统资源分配的基本单位，线程是 CPU 调度和分派的基本单位。

③它们的基本状态都是就绪、运行、等待。

④它们都具有创建、撤销、切换这些操作，但是线程的时空开销更小。

⑤进程的存在标志是进程控制块 PCB，线程的存在标志是 TCB，进程的 PCB 会关联其内部线程的 TCB。

⑥有单进程单线程、单进程多线程、多进程单线程、多进程多线程这些关系。

2.27 处理器调度的层次

在多道程序环境下，一个作业从提交到执行，通常都要经历多级调度，如高级调度、低级调度、中级调度等。

三个层次：

高级调度（长程调度）、中级调度（中程调度）、低级调度（短程调度）。

低级调度又分为抢占式（剥夺式）和非抢占式（非剥夺式），抢占式的开销更大，一般内核关键程序是非抢占式的，用户进程是抢占式的。

举几个例子，从外存后备队列组织一批作业进内存（就绪队列或挂起就绪）是高级调度，从就绪队列里面挑一些作业（进程）来占用处理器，从就绪到运行，是低级调度，把活动就绪和活动等待的放到外存变成静止就绪和静止阻塞，选一个激活，实现这种“对换”，叫中级调度。

作业调度是高级调度，进程调度是低级调度。

注意，作业被调度进入主存储器，并以进程的形式存在，其状态就是执行状态。处于执行状态的作业并不意味着一定在 CPU 上运行，是否运行依赖于进程控制，处于执行状态的作业可以有多个。

2.28 选择调度方式和算法的准则

①面向用户的准则：

周转时间短、响应时间快、截止时间的保证、优先权准则

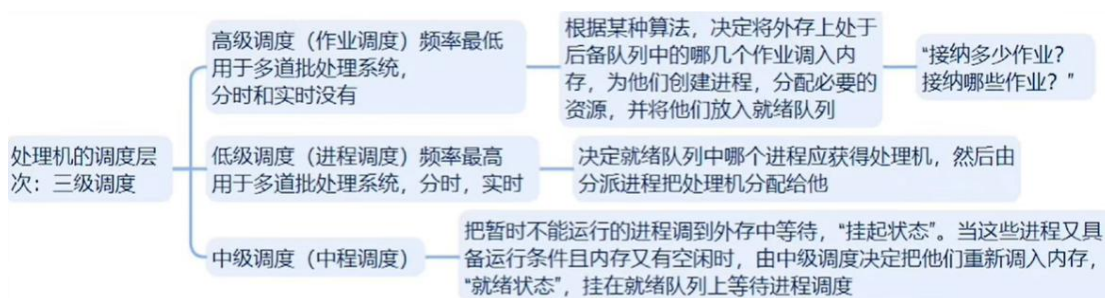
②面向系统的准则：

系统吞吐量、处理机利用率好、各类资源平衡利用。

③最优准则：

最大的 CPU 利用率、最大的吞吐量、最短的周转时间、最短的等待时间、最短的响应时间。

2.29 调度算法



①先来先服务 FCFS（非抢占式）

不利于短作业而优待长作业，使短作业的周转时间变得很大，从而平均周转时间也变大。

②最短作业优先 SJF（非抢占式）

对长作业不利，可能会出现饥饿。但是能对短作业提供良好的响应时间，提高吞吐量。

③最短剩余时间优先 SRTF（抢占式）

提高吞吐量，对长作业不利，可能会出现饥饿。

④最高响应比优先 HRRF，是 FCFS 和 SJF 的折中，具有良好的均衡性（非抢占式）

优待短作业，短作业容易得到高响应比；但是长作业在系统中等待的时间越长，响应比越大，优先级越高。

衡量调度算法优劣的因素：

响应时间、周转时间、资源利用率、作业吞吐率、公平性。

2.30 周转时间

周转时间指的是作业从提交到完成所花费的时间，由驻外等待调度时间、驻内等待调度时间、执行时间、阻塞时间组成。

3 同步、通信与死锁

3.1 顺序程序设计

具有执行的顺序性、环境的封闭性、结果的确定性、过程的可再现性，优点是给程序的编制和调试带来很大方便，缺点是计算机系统效率不高。

3.2 程序并发执行

程序的并发性是指一组程序的执行在时间上是重叠的，即多个程序都处于已经开始执行但尚未执行完成的状态。

从宏观上看，并发性反映一个时间段中几个进程都在同一处理器上，处于运行还未运行结束状态；从微观上看，任一时刻仅有一个进程在处理器上运行。

并发的实质是一个处理器在多个进程之间的多路复用，并发是对有限的物理资源强制行使多用户共享，以提高系统资源利用率。

并发程序设计是指，一个程序被分成若干可同时执行的小程序，每个小程序及其处理的数据组成一个进程。

并发程序设计的特征：

并行性、共享性、制约性、交互性

并发进程的分类：

无关的、交往的

并发进程的无关性是进程执行与时间无关的充分条件（Bernstein 伯恩斯坦条件）。

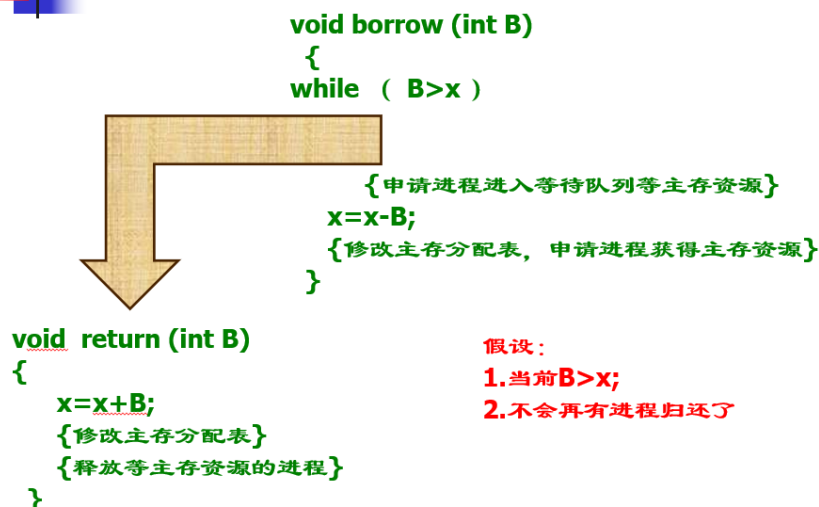
而对于两个交往的并发进程，执行的相对速度无法相互控制，各种与时间有关的错误就可能产生。与时间有关错误的有两种表现形式：结果不唯一和永远等待。



原因：对临界资源 Aj 的访问没有加以控制。

- 系统中当前剩余票数为13张，（按照前述）2个并发的售票进程卖出2张票后，系统中可能还剩余（ ）
- A 11
 - B 11, 12, 13
 - C 12, 11
 - D 13, 12

(永远等待) 内存管理问题



原因: 申请资源的进程进入等待队列之前就有另一个进程来释放归还资源了, 但前一个资源还未成为等待者, 另一个进程的释放相当于空操作, 因此申请资源的进程就永远等待了。

3.3 进程同步

在多道程序设计系统中, 同一时刻可能存在多个进程并发或并行地执行, 它们之间存在两种基本关系: 竞争和协作。

① 竞争关系

系统中的多个进程之间彼此无关, 但是它们由于竞争共享资源而产生了联系, 这是一种间接制约关系, 又称互斥关系。资源的竞争会引发两个问题, 一个是死锁, 一组进程因为争夺资源而陷入永远等待, 另一个是饥饿, 一个可运行的进程因为其他进程总优先于它而迟迟不能被执行, 解决饥饿问题最简单的策略就是 FCFS。

要保证多进程能互斥地访问共享资源, 就要解决死锁和饥饿的问题。

进程互斥是指, 多个进程由于相互争夺独占型资源而产生的竞争制约关系, 若干进程要使用同一共享资源时, 任何时刻最多允许一个进程使用, 其他进程必须等待, 直到占有资源的进程释放该资源。

② 协作关系

系统中的多个进程之间彼此相关, 这是一种直接制约关系。

进程同步是指, 为完成共同任务的并发进程基于某个条件来协调其活动, 由于需要在某些位置上排定执行的先后顺序而等待、传递信息而产生的协作制约关系。一个进程的执行依赖于协作进程的消息或信号, 当一个进程没有得到来自于协作进程的消息或信号时需等待, 直到消息或信号到达才被唤醒。

进程互斥关系是一种特殊的进程同步关系, 也是对使用资源次序的一种协调。

3.4 临界区 (critical section, 简称 CS)

并发进程中与共享变量有关的程序段。

把临界区的调度原则 (同步机制应遵循的原则、进程进入临界区的调度原则) 总结成几句话: 空闲让进、忙则等待、择一而入、算法可行。

3.5 临界资源

共享变量所代表的资源, 即一次仅能供一个进程使用的资源。也叫共享变量或互斥资源。

3.6 临界区的管理

要解决临界区（进程互斥）问题的方法，既有硬件方法（关中断、测试并设置指令、对换指令）又有软件方法（Peterson 算法），但是最好用的还是信号量以及原语（P、V 操作）。

注意，原语是机器指令的延伸，往往是为完成某些特定的功能而编制的一段系统程序。原语操作也称做“原子操作”，即一个操作中的所有动作要么全做，要么全不做。执行原语操作时，要屏蔽中断，以保证其操作的不可分割性。

3.7 信号量与 PV 操作（Dijkstra 发明的同步工具）

①同步和同步机制

常见的同步机制有信号量及 PV 操作、管程、消息传递。

②信号量

整型信号量只能是非负整数，给它赋值只有三种方法：赋初值、P 操作、V 操作。

1、整型信号量

- **整型信号量**——非负整数，除了初始化外，只能通过两个原子操作wait和signal (P, V) 来访问。

- **wait和signal操作描述：**

wait(S): while $S \leq 0$ do no-op 测试有无可用资源

$S := S - 1$; 可用资源数减一个单位

signal(S): $S := S + 1$;

- **主要问题：**只要 $S \leq 0$ ，wait操作就不断地测试（盲等），因而，未做到“让权等待”。

2、记录型信号量

- **基本思想**

- 1、设置一个代表资源数目的整型变量value（资源信号量）
- 2、设置一链表L用于链接所有等待的进程

- **记录型信号量的数据结构**

```
Type semaphore = record
    value: integer;
    L: list of process;
end
```

信号量的物理含义：

$S > 0$ 表示有 S 个资源可用

$S = 0$ 表示无资源可用

$S < 0$ 则 $|S|$ 表示 S 等待队列中的进程个数

P、V 操作的含义：

P(S)表示申请一个资源，将信号量的值减 1，大于等于 0 则继续执行，小于零则表示申请不到了，原来最多是 0 个，该执行 P 操作的进程被阻塞，放入与信号量有关的队列中。

V(S)表示释放一个资源，将信号量的值加 1，大于 0 则继续执行，小于等于 0 则从队列中释放一个进程，将其转至就绪态。

在一定的条件下，P 操作代表挂起进程，V 操作代表唤醒被挂起进程。

③信号量和 PV 操作可以解决进程互斥问题

在每个程序中用于实现互斥的 $P(mutex)$ 和 $V(mutex)$ 必须成对出现，即先做 P 操作，进入临界区；后做 V 操作，退出临界区。

注意，互斥信号量 $mutex$ 的初值一般为 1。

当有进程在临界区中时， $mutex$ 的值为 0 或负值，否则 $mutex$ 值为 1，因为只有一个进程，可用 P 操作把 $mutex$ 减至 0，故可保证互斥操作，这时试图进入临界区的其他进程会因执行 $P(mutex)$ 而被迫等待。 $mutex$ 的取值范围是 $1 \sim -(n-1)$ ，表明有一个进程在临界区内执行，最多有 $n-1$ 个进程在信号量队列中等待。

一、用信号量解决「互斥问题」（比如“抢厕所”）

互斥的核心是：一个资源同一时间只能被一个人用（比如厕所只有 1 个坑位，不能同时进 2 个人）。

步骤：

1. 先定义一个信号量 s ，初始值设为 1（代表“这个资源最多同时被 1 个进程用”）；
2. 在“要使用资源的代码段（临界区）前后”加 P、V 操作：
 - 进入厕所前（要使用资源）：执行 $P(s)$ （申请占用厕所）；
 - 离开厕所后（用完资源）：执行 $V(s)$ （释放厕所）。

举例（2 个人抢 1 个厕所）：

- 初始 $s=1$ （厕所空闲）；
- 甲想上厕所：执行 $P(s) \rightarrow s=0$ （申请成功），甲进厕所（用资源）；
- 这时候乙想上厕所：执行 $P(s) \rightarrow s=-1$ （申请失败），乙去排队（进程阻塞）；
- 甲上完厕所：执行 $V(s) \rightarrow s=0$ （释放资源），发现 $s \leq 0$ （有人排队），叫乙进厕所；
- 乙上完厕所：执行 $V(s) \rightarrow s=1$ （厕所回到空闲状态）。

二、用信号量解决「同步问题」（比如“做饭-吃饭”）

同步的核心是：必须先做完 A，才能做 B（比如必须等饭做好了，才能吃饭，不能没做饭就先吃）。

步骤：

1. 先定义一个信号量 s ，初始值设为 0（代表“初始时 B 的条件没满足”）；
2. 在前驱操作（A）的末尾加 V 操作（做完 A，告诉 B “可以开始了”）；
3. 在后驱操作（B）的开头加 P 操作（做 B 前，先检查 A 有没有做完，没做完就等）。

举例（做饭-吃饭）：

- 定义信号量 $s=0$ （初始时“饭没做好”）；
- 「做饭（前驱）」的流程：
淘米→炒菜→把饭做好 → 执行 $V(s)$ （告诉“吃饭”：饭好了）；
- 「吃饭（后继）」的流程：
执行 $P(s)$ （检查饭做好没：初始 $s=0 \rightarrow$ 执行 $P(s)$ 后 $s=-1$ ，去排队等饭）；
等做饭的人执行 $V(s) \rightarrow s=0$ （饭好了），开始吃饭。

总结

- 互斥：一个信号量 + 临界区前后加 P、V（管“不能同时用”）；
- 同步：一个信号量 + 前驱加 V、后继加 P（管“必须先做 A 再做 B”）。

- 为安全以及管理方便，某商场最多容许500人在其中游览或消费等。每个人必须在进门处的登记本上留下个人（手机号码以及姓名）信息，游览或消费后从东门离开。试用P、V原语描述该行为过程。
- Semaphore sc=500,huchi=1;//2 分
- P(sc);//2 分
- 进入;
- P(huchi);///2分，排队登记
- 登记手机号、名字信息;
- V(huchi);//2 分
- 游览或消费;
- 离开;
- V(sc);//2 分

3.8 管程



3.4 管程机制

✋ 解决方法----管程 (Monitors)

Dijkstra于1971年提出，为每个共享资源设立一个“秘书”来管理对它的访问。一切来访者都要通过秘书，而秘书每次仅允许一个来访者（进程）来访问共享资源。这样既便于系统管理共享资源，又能保证进程的互斥访问和同步。1973年，Hanson和Hoare又把“秘书”概念发展为管程概念。

① 管程的定义

一个数据结构和在该数据结构上能被**并发进程所执行的一组操作**，这组操作能**同步进程和改变管程中的数据**。

3.9 进程通信

并发进程之间的交互必须满足两个基本要求：同步和通信。

进程通信指的是进程之间的信息交换。

信号通信机制有三种：

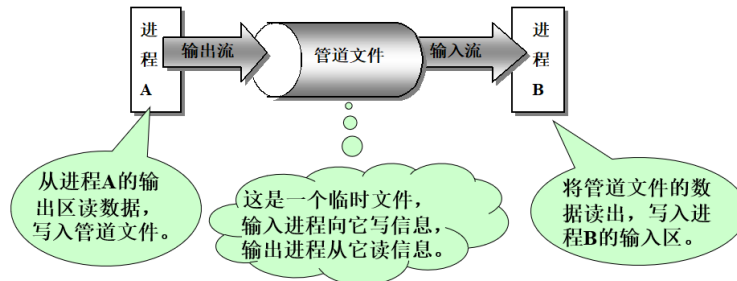
①管道通信机制

管道是单向的，管道的读写逻辑是先进先出的。在Linux系统中，管道利用文件系统的file结构和inode节点来实现。当创建管道时，在系统打开文件表中建立管道文件的两个file结构分别用于控制管道的写操作和读操作。两个file结构指向同一个inode，inode指向磁盘上的物理页面。内核调用pipe()创建管道后，返回句柄files[0]和files[1]，读者进程通过files[0]从管道取出数据，通过files[1]往管道写入数据。

3.5 进程通信—高级通信

3. 管道(Pipe)通信

共用一个管道文件，进程之间的关系有：
互斥关系——输出和输入进程不可能同时读或者写；
同步关系——当管道文件为空时，输入进程等待输出进程，当管道文件为满时，输出进程等待输入进程。



②共享内存通信机制

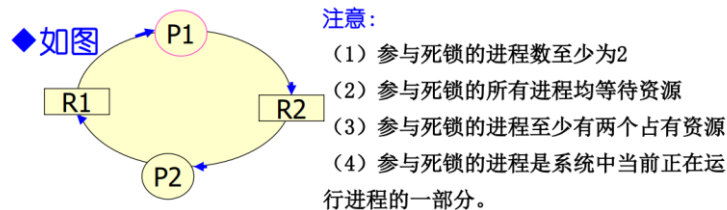
③消息传递通信机制

又分为直接通信方式（消息缓冲通信）和间接通信方式（信箱通信方式）。

3.10 死锁

①死锁的定义

多个进程在运行过程中争夺资源、推进顺序不当或进程通信而相互等待对方的资源，处于都不能执行而永远等待的僵局，若无外力作用，这些进程都将无法向前推进。



②产生死锁的四个必要条件与死锁的预防

破坏这四个条件之一，就可以防止死锁。

互斥条件：

临界资源是独占资源，进程应互斥且排他地使用这些资源。

破坏它一般行不通。

请求和保持条件：

进程在请求资源而得不到满足时，不释放已占有的资源。

可以采用**预先分配**的策略来破坏，系统要求任一进程必须预先申请它所需的全部资源，而且仅当该进程的全部资源要求能得到满足时，系统才能给予一次性分配，然后启动该进程运行，但是在分配时只要有一种资源不满足，系统就不会给进程分配资源。进程运行期间，不会再请求新的资源。缺点：资源严重浪费、进程延迟进行。

不剥夺条件：

不可抢占，已获得的资源只能由进程自愿释放而不能被其他进程剥夺。

一个已经保持了某些资源的进程，当它再提出新的资源要求而不能立即得到满足时，必须释放它已经保持的所有资源，待以后需要时再重新申请。实现比较复杂，且要付出很大代价；此外，还因为反复地申请和释放资源，而使进程的执行无限地推迟，延长了周转时间，增加了系统的开销，降低了系统吞吐量。（例如打印机打印的结果不连续）

循环等待条件：

又称环路条件，有循环等待链。

可以采用**有序分配**的策略来破坏，把系统中所有资源编号，进程在申请资源时必须按资源编号的递增次序进行，否则操作系统不予分配。优点：资源利用率和系统吞吐量与另两种方法相比有较明显的改善。缺点：为系统中各种类型的资源所分配的序号必须相对稳定，这就限制了新设备类型的增加；作业实际使用资源的顺序与系统规定的顺序不同而造成资源的浪费；限制了用户简单、自主地编程。

四、处理死锁的基本方法

☞ 不让死锁发生

❖ **预防死锁**（静态策略）

❖ **避免死锁**（动态策略）

☞ 让死锁发生

❖ **检测死锁**

❖ **解除死锁**



③ 死锁的避免

各种死锁防止方法能够防止发生死锁，但是必然会降低系统的并发性，导致资源利用率低。死锁避免却与之相反，允许系统中同时存在前三个必要条件，只是要确保不会出现进程循环等待链，从而避免死锁。

系统运行过程中，允许进程**动态地**申请资源，但系统在进行资源分配之前，应先计算此次资源分配的安全性。若此次分配不会导致系统进入不安全状态，则将资源分配给进程；否则，令进程等待。

最具有代表性的**避免死锁算法**是**银行家算法**（Banker's Algorithm）。

银行家算法的两个条件：分批提出请求，request 代表某一个进程某一次提出的资源请求，提出请求的资源 request **不能超过**银行家所拥有的资源的最大值 Available **也不能超过**自己的 Need，如果不超过，那么银行家答应分配给它，先假设分配给它，判断安全性，进而决定能否分配，如果能分配，当该进程得到所有资源的时候，全部还给银行家。

④ 死锁的检测与恢复（解除）

死锁的检测与恢复往往配套使用。死锁检测算法采用类似银行家算法的数据结构。解除死锁的常用方法有系统重启法、资源剥夺法、进程回退法，进程撤销法。

总之，**解决死锁问题有三种策略和方法**：死锁防止、死锁避免、死锁检测和解除。死锁的**防止**是指系统预先确定资源分配策略，进程按此规定来申请和使用资源，保证死锁的一个必要条件不会被满足，使得系统不发生死锁；其**缺点是资源利用率低，或对资源使用的限制过严**。死锁的**避免**涉及动态分析和检测新的资源请求和资源分配情况，以确保系统始终处于安全状态，**放宽资源的使用条件**，银行家算法是著名的死锁避免算法，但缺乏实用价值。死锁的**检测和解除**表明操作系统总是**同意对资源的申请，对资源的分配不施加任何限制，允许系统发生死锁**，但必须建立检测机制选择**安全检测算法，周期性地检测是否发生死锁**，若有则将其检测出来再**采取措施解除死锁**。虽然死锁检测对于资源的使用不施加任何限制，但其实现开销较大。

3.11 系统的安全状态

如果系统能按一个安全序列，为每个进程分配其所需的资源，直至每个进程都能顺利地完成任务，称系统处于**安全状态**。若不存在，则称系统处于**不安全状态**。

安全状态一定是没有死锁发生的，但是进入不安全状态不等同于死锁，不安全状态里面包含了死锁的可能。

避免死锁的实质就是要确保系统不进入不安全状态。

3.12 死锁的检测与恢复（解除）

死锁的检测与恢复往往配套使用。死锁检测算法采用类似银行家算法的数据结构。解除死锁的常用方法有系统重启法、资源剥夺法、进程回退法，进程撤销法。

补充：

进程的低级通信机制主要有原子操作、锁机制、信号量和 PV 操作及管程；进程的高级通信机制有消息传递、共享内存和管道机制等。

4 存储管理

4.1 程序的装入和链接

一个用户源程序要变为在内存中可执行的程序，通常要进行编译、链接、装入的三步处理。

编译：

由编译程序将用户源程序编译成若干个目标模块。

链接：

由链接程序将目标模块和相应的库函数链接成装入模块。

根据链接时间的不同，可将链接分成三种：

- ❖ 静态链接
- ❖ 装入时动态链接
- ❖ 运行时动态链接

装入：

由装入程序将装入模块装入内存。

- ❖ 绝对装入方式
- ❖ 可重定位装入方式
- ❖ 动态运行时装入方式

绝对装入方式不需要对程序和数据的地址进行修改，只适用于单道程序环境。

可重定位装入方式实际上是地址变换或地址映射的过程，有三种地址重定位的方式：静态地址重定位、动态地址重定位、运行时链接地址重定位，适用于多道程序环境。

4.2 地址转换

原因：

当程序装入内存时，操作系统要为该程序分配一个合适的内存空间，由于程序的逻辑地址与分配到内存物理地址不一致，而 CPU 执行指令时是按物理地址进行的，所以要进行地址转换。

4.3 连续分配存储管理方式

连续分配是指，为一个用户程序分配一片连续的内存空间。

- ❖ 4.2.1 单一连续分配方式
- ❖ 4.2.2 固定分区分配方式
- ❖ 4.2.3 动态分区分配方式
- ❖ 4.2.4 动态重定位分区分配方式
- ❖ 分区的存储保护
- ❖ 覆盖与交换

①单一连续分配方式

采用静态分配方式，即作业一旦进入内存，就要等待它运行结束后才能释放内存。内存中只装入一道作业运行，内存空间浪费大，各类资源的利用率也不高。

②固定分区分配方式

最早使用的一种可运行多道程序的存储管理方法，将内存空间划分为若干个固定大小的分区，除 OS 占一区外，其余的一个分区装入一道程序。分区的大小可以相等，也可以不等，但事先必须确定，在运行时不能改变。适用于程序大小和出现频率已知情况。

③动态分区（可变分区）分配方式

内存不是预先划分好的，作业装入时，根据作业的需求和内存空间的使用情况来决定是否分配：若有足够的空间，则按需要分割一部分分区给该进程；否则令其等待内存空间，提高了主存的利用率。

常用的分区分配算法有：首次适应算法、循环首次适应算法、最佳适应算法、最坏适应算法、快速适应算法。（最先适应算法、下次适应算法）

碎片问题：

在分区存储管理方式中，必须把作业装入到一片连续的内存空间。如果系统中有若干个小分区，其总容量大于要装入的作业，但由于它们不相邻接，也将致使作业不能装入内存。内部碎片是指，分配给作业的存储空间中未被利用的部分，例如固定分区中存在的碎片。外部碎片是指，系统中无法利用的小的空闲分区，例如动态分区中存在的碎片。

④动态重定位分区分配技术

在动态分区分配算法中增加拼接功能，在找不到足够大的空闲分区来满足作业要求，而系统中总空闲分区容量可以满足作业要求时，进行拼接。

分区的存储保护：

存储保护是为了防止一个作业有意或无意地破坏操作系统或其它作业，有界限寄存器和存储保护键两种方法。

4.4 内存不足的存储管理技术

覆盖与交换技术都是在多道程序环境下用来扩充内存的两种方法。

①移动技术

给内存中的进程“搬新家”。

②覆盖技术

把一个程序划分为一系列功能相对独立的程序段（称为覆盖），让执行时并不要求同时装入内存的覆盖组成一组（称为覆盖段），共享主存的同一个区域，从而解决在小的存储空间中运行大作业的问题。在同一个作业或进程中进行。

③对换技术

将暂时不用的某个进程及数据（首先是处于阻塞状态优先级最低的）部分（或全部）从内存移至外存（备份区或对换区，采用连续分配的动态存储管理方式）中去，让出内存空间，同时将某个需要的进程调入到内存中，让其运行。交换到外存的进程需要时可以被再次交换回（选择换出时间最久的）内存中继续执行。在作业或进程之间进行。

4.5 基本分页存储管理方式

为了解决碎片问题，采取离散分配方式，分别为分页式存储管理、分段式存储管理、段页式存储管理。

在分页存储管理方式中，如不具备页面对换功能，不支持虚拟存储器功能，在调度作业运行时，必须将它的所有页面一次调入内存，若内存没有足够的块，则作业等待，这种存储管理方式称为纯分页或基本分页存储管理方式。

①空间划分

将一个用户进程的地址空间（逻辑）划分成若干个大小相等的区域，称为页或页面，并为各页从 0 开始编号。

将内存空间也分成若干个与页大小相等的区域，称为（存储、物理）块或页框（frame），同样从 0 开始编号。

②内存分配

在为进程分配内存时以块为单位，将进程中若干页装入到多个不相邻的块中，最后一页常装不满一块而出现页内碎片。

一、基本思想

- 页面大小-----由地址结构（逻辑）决定
- 若页面较小：
 - ✧ 减少页内碎片和内存碎片的总空间，有利于提高内存利用率。
 - ✧ 每个进程页面数增多，从而使页表长增加，占用内存就较大。
 - ✧ 页面换进换出速度将降低。
- 若页面较大：
 - ✧ 每个进程页面数减少，页表长度减少，占用内存就较小。
 - ✧ 页面换进换出速度将提高。
 - ✧ 会增加页内碎片不利于提高内存利用率。

页号	位移量（页内地址）
----	-----------

总之，页面大小的选择适中就好。

③页表

页表一次，内存一次，一共 2 次访存。

- 为了便于在内存找到进程的每个页面所对应块，系统为每个进程建立一张页面映象，简称页表，如图。
 - 记录了页面在内存中对应的块号
 - 页表一般存放在内存中
 - 页表的基址及长度由页表寄存器给出
 - 访问一个字节的数据/指令需访问内存2次(页表一次,内存一次)，所以出现内存访问速度降低的问题。

作业的地址空间

页表

内存空间

页表始址 页表长度

💥

④地址结构

- 逻辑地址:

31	12	11	0
页号p		位移量w	

 - 地址长为32位，其中0-11位为页内地址，即每页的大小为 $2^{12}=4KB$;
 - 12-31位为页号，地址空间最多允许有 $2^{20}=1M$ 页。
- 物理地址:

21	12	11	0
块号b		块内位移d	

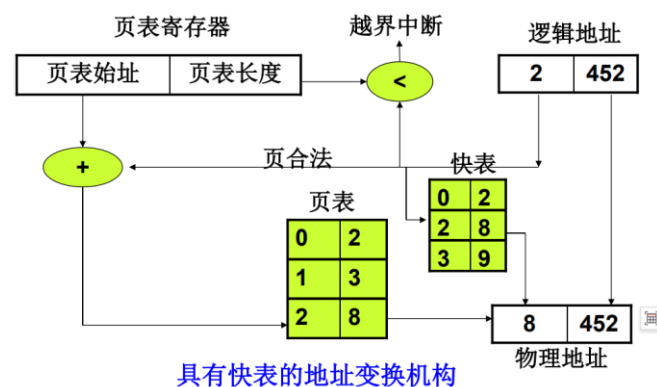
 - 地址长为22位，其中0-11位为块内地址，即每块的大小为 $2^{12}=4KB$ ，与页相等;
 - 12-21位为块号，内存地址空间最多允许有 $2^{10}=1K$ 块。

⑤地址变换机构

为了能将用户地址空间中的逻辑地址变换为内存空间中的物理地址,在系统中必须设置地址变换机构,分为基本的地址变换机构和具有快表的地址变换机构。

具有快表的地址变换机构:

- 基本的地址变换机构存在的问题
 - 地址变换速度降低（因页表放于内存中，CPU访问一个字节的数据需两次访问内存）
- 目的：为提高地址变换速度
- 快表（相联寄存器、相联存储器、TLB）
 - 为一种特殊的高速缓冲存储器。
 - 内容-----页表中的一部分或全部内容。
 - CPU产生逻辑地址的页号，首先在快表中寻找，若找到(命中)就找出其对应的物理块；若未找到(未命中)，再到页表中找其对应的物理块，并将之复制到快表。
 - 若快表中内容满，则按某种算法淘汰某些页。



有效访问内存的时间

$$T = P_{TLB} * (T_{TLB} + T_M) + (1 - P_{TLB}) * (T_{TLB} + 2T_M)$$

其中, P_{TLB} 为快表的命中率, T_{TLB} 为快表的访问时间, T_M 为内存的访问时间

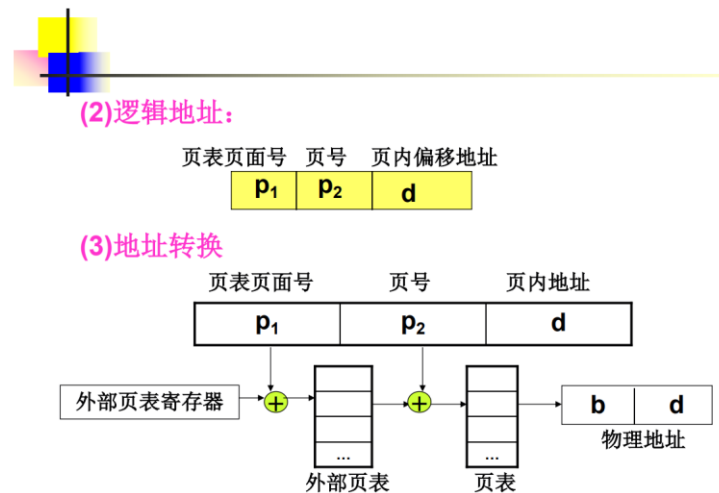
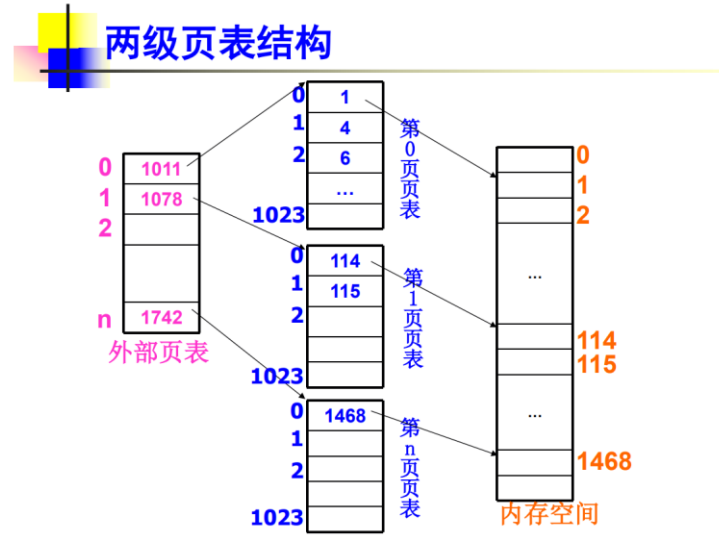
⑥多级页表

若逻辑地址空间很大,则划分的页就很多,页表就很大,其占用的存储空间(要求连续)就大,实现较难。

解决问题的方法: 只将当前需用的部分页表项调入内存,其余的需用时再调入; 多级页表,将页表再进行分页,并离散地将各个页表页面存放在不同的物理块中,同时也再建立一张页表(外层页表)用以记录页表页面对应的物理块号。

❖ 二级页表

(1) 将页表再进行分页，并离散地将各个页表页面存放在不同的物理块中，同时也再建立一张页表（外层页表）用以记录页表页面对应的物理块号。



❑ 多级页表

➤ 将外层页表再进行分页，也将各外层页表页面离散地存放在不同的物理块中，再利用第2级的外层页表来记录它们之间的对应的关系。

➤ 逻辑地址：

外层页表页面号	页表页面号	页号	页内偏移地址
p_1	p_2	p_3	d

快表可以加快地址映射，多级页表可以减少内存空间占用。

⑦ 页的共享与保护

共享代码（数据）的实现方法：由各进程共享的一段代码（数据），要求各进程相应的页存入内存相同物理块中。

带来的问题，若共享数据与不共享数据划在同一块中，则有些不共享的数据也被共享，不易保密，而且计算共享数据的页内位移较困难。

实现数据共享的最好方法---段式存储管理。

但是，页式存储管理系统也提供了两种保护方式：地址越界保护、在页表中设置保护位（定义操作权限：只读，读写，执行等）。

4.6 基本分段存储管理方式



分段系统的基本原理——段表

- 记录了段与内存位置的对应关系
- 段表常保存在内存中（一组寄存器）
- 段表的基址及长度由段表寄存器给出
- 访问一个字节的的数据/指令需访问内存两次(段表一次，内存一次)，所以也出现内存访问速度降低的问题。
- 逻辑地址：

段号	段内地址
----	------

段号	段长	基址
0	30k	40k
1	20k	80k
2	15k	120k
3	10k	150k

段表基址

段表长度

将用户作业的逻辑地址空间，依据逻辑信息的相对完整来划分成若干个大小不等的段。各段有段名（常用段号代替），首地址为 0。

在为作业分配内存时以段为单位，分配一段连续的物理地址空间；段间不必连续。

记录了段与内存位置的对应关系，段表常保存在内存中（一组寄存器，段表的基址及长度由段表寄存器给出，访问一个字节的的数据/指令需访问内存两次(段表一次，内存一次)，所以也出现内存访问速度降低的问题。

注：整个作业的逻辑地址空间是二维的，其逻辑地址由段号和段内地址组成；物理地址空间是一维的。需要 CPU 的硬件支持（地址变换机构）。

4.7 分页和分段的区别

分段是为了满足用户模块化程序设计的需要，内存的分配单位是段，是信息的逻辑单位，由源程序的逻辑结构及含义决定，是用户可见的，段长由用户根据需要来确定，段起始地址可以从任何内存地址开始。段式的作业地址空间是二维的。优点是更好地实现了数据的共享与保护，段长可以动态增长，便于动态链接。

分页是为了实现离散分配并提高内存利用率，内存的分配单位是页，是信息的物理单位，与源程序的逻辑结构无关，是用户不可见的，页长由系统来确定，是固定的，页面只能从页大小的整数倍地址开始。页式的作业地址空间是一维的。页式的优点是有效解决了碎片问题，有效提高内存利用率。

段页式存储管理是分段和分页原理的结合，先将用户程序分成若干个段，并为每一个段赋一个段名，再把每个段分成若干个页。

其地址结构由

段号	段内页号	页内位移
----	------	------

组成。

段页式系统中的地址变换

- 系统中设段表和页表，均存放于内存中。读一字节的指令或数据须访问内存三次。为提高执行速度可增设高速缓冲寄存器。
- 每个进程一张段表，每个段一张页表。
- 段表含段号、页表始址和页表长度。页表含页号和块号。
- 进行地址变换：
先用段号与段寄存器中的段长进行比较，若小于段长则利用段表始址和段号找出该段页表的始址，(否则越界中断)，再用逻辑地址中的段内页号在页表中找到相应的块号最后与页内位移形成物理地址。

4.8 虚拟存储器

记住，像上面加了“基本”两个字的就不具备虚拟存储器。

①常规存储器管理方式的特征：一次性、驻留性。

常规存储器要求一个作业全部装入内存后方能运行。导致(1)有的作业很大，所需内存空间大于内存总容量，无法运行。(2)有大量作业要求运行，但内存容量不足以容纳下所作业，只能让一部分先运行，其它在外存等待。

解决方法：一方面，可以增加内存容量；另一方面，可以从逻辑上扩充内存容量（覆盖、对换、虚拟存储器）。

一次性：作业在运行前需一次性地全部装入内存。

驻留性：作业装入内存后，便一直驻留内存，直至作业运行结束。

②局部性原理

指程序在执行时呈现出局部性规律，即在一较短时间内，程序的执行仅限于某个部分，相应地，它所访问的存储空间也局限于某个区域。

局部性又表现为时间局部性(由于大量的循环操作，某指令或数据被访问后，则不久可能会被再次访问)、空间局部性（如顺序执行，指程序在一段时间内访问的地址，可能集中在一定的范围之内）。

③虚拟存储器的定义

仅把作业的一部分装入内存便可运行作业的存储管理系统，它具有请求调入功能和置换功能，能从逻辑上对内存容量进行扩充，其逻辑容量由外存容量和内存容量之和决定，其运行速度接近于内存，成本接近于外存。

基于局部性原理，程序在运行之前，没有必要全部装入内存，仅须将当前要运行的页（段）装入内存即可。运行时，如访问的页（段）在内存中，则继续执行，如访问的页未在内存中（缺页或缺段），则利用 OS 的请求调页（段）功能，将该页（段）调入内存。如内存已满，按某种置换算法将内存中的某页（段）调至外存，从而调入需访问的页（段）。

一个计算机系统的虚拟存储器的最大容量是由计算机的地址结构确定的，其实际容量是由内存和硬盘容量之和确定的。

④虚拟存储器的实现方法

基于分页请求或者分段请求，在此基础上增加请求调页（段）功能、页面（分段）置换功能，形成页式（段式）虚拟存储器系统。

⑤虚拟存储器的特征

多次性、对换性、虚拟性

1、多次性

- 多次性是虚拟存储器最重要的特征，指一个作业被分成多次调入内存运行。

2、对换性

- 对换性指允许在作业运行过程中进行换进、换出。换进、换出可提高内存利用率。

3、虚拟性

- 虚拟性是指能够从逻辑上扩充内存容量，使用户所看到的内存容量远大于实际内存容量。虚拟性是虚拟存储器所表现出来的最重要的特征，也是实现虚拟存储器最重要的目标。

- 注：虚拟性以多次性和对换性为基础，而多次性和对换性又以离散分配为基础。



4.9 请求分页虚拟存储管理

硬件支持：

①页表机制

②缺页中断机构

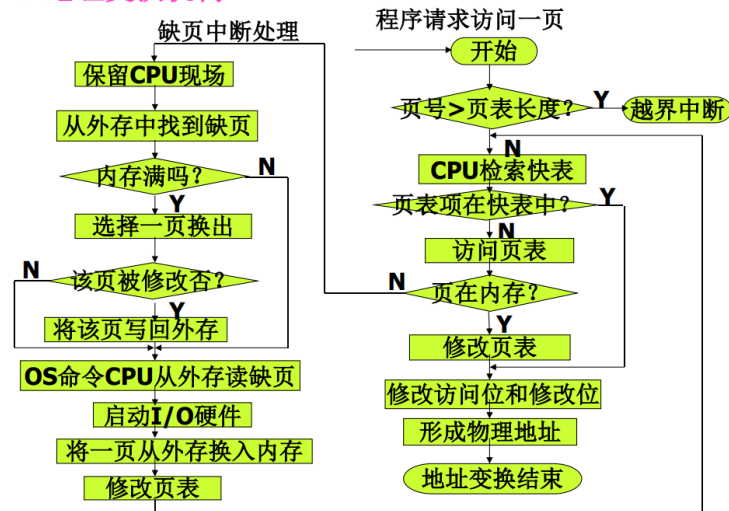
在请求分页系统中，当访问的页不在内存，便产生一缺页中断，请求 OS 将所缺页调入内存空闲块，若无空闲块，则需置换某一页，同时修改相应页表表目。

缺页中断与一般中断的区别：

- (1) 在指令执行期间产生和处理中断信号
- (2) 一条指令在执行期间，可能产生多次缺页中断

③地址变换机构

3、地址变换机构



程序请求访问某页时，先检查页号是否越界，若正常则优先检索快表，命中则直接生成物理地址；未命中则查页表，若页在内存则更新快表信息后形成物理地址。若页不在内存，则触发缺页中断：先保留 CPU 现场，从外存定位缺页，若内存已满则选页换出，再将缺页读入内存并更新页表，回到地址变换流程形成物理地址。

④内存分配

为进程分配页框的策略有两种：固定分配、可变分配。

页面替换的策略有两种：局部替换、全局替换。

物理块的分配策略有固定分配局部置换、可变分配全局置换、可变分配局部置换。

⑤页面调入策略

预调页、请求调页

三、请求分页中的页面调入策略

- 调入策略决定**什么时候**将一个页面由外存调入内存，从**何处**将页面调入内存。
 - **何时调入页面**
 - ❖ **预调页策略**：将那些**预计**在不久便被访问的页预先调入内存。这种调入策略**提高了调页的效率，减少了I/O次数**。但由于这是一种基于局部性原理的预测，若预调入的页面在以后很少被访问，则造成浪费，故这种方式**常用于程序的首次调入**。
 - ❖ **请求调页策略**：当进程运行中访问的页出现缺页时，则发出缺页中断，提出请求调页，由**OS**将所需页调入内存。这种策略实现简单，应用于目前的虚拟存储器中，但易产生较多的缺页中断，且每次调一页，系统开销较大，容易产生抖动现象。
 - **从何处调入页面**

在请求分页系统中，通常将外存分成了文件区和对换区，文件区按**离散分配**方式存放文件，对换区按**连续分配**方式存放对换页。

⑥页面置换算法

页面置换算法也称为页面淘汰算法，是用来选择换出页面的算法。

页面置换算法的优劣直接影响到系统的效率，若选择不合适，可能会出现以下现象：刚被淘汰出内存的页面，过后不久又要访问它，需要再次将其调入，而该页调入内存后不久又再次被淘汰出内存，然后又要访问它，如此反复，使得系统把大部分时间用在了页面的调进换出上，而几乎不能完成任何有效的工作，这种现象称为**抖动**（又称**颠簸**）。

预防方法

- ❖ 选择合适的局部置换策略
- ❖ 在**CPU**调度程序中引入工作集算法
- ❖ 挂起若干进程

降低缺页率防止抖动

- 1、增加分配给进程的页框数
- 2、选择合适的页面置换算法
- 3、改进应用程序结构

(1) 最佳置换算法

选择永远不再需要的页面或最长时间以后才需要访问的页面予以淘汰。

(2) 先进先出置换算法

选择先进入内存的页面予以淘汰。

(3) 最近最久未使用置换算法

选择最近一段时间最长时间没有被访问过的页面予以淘汰。

(4) CLOCK 置换算法

该算法是 LRU 和 FIFO 的折衷。该算法要求为每页设置一个访问位，并将内存中的所有页链接成一个循环队列。当某页被访问时，系统将其访问位设置为 1。置换时采用一个指针，从当前指针位置开始按序检查各页，若访问位为 0 则选择该页换出，若访问位为 1

则将其设置为 0，最后指针停留在被置换页的下一页上。



(1) 简单Clock置换算法

- 该算法是LRU和FIFO的折衷。

该算法要求为每页设置一个访问位，并将内存中的所有页链接成一个循环队列。当某页被访问时，系统将其访问位设置为1。置换时采用一个指针，从当前指针位置开始按序检查各页，若访问位为0则选择该页换出，若访问位为1则将其设置为0，最后指针停留在被置换页的下一页上。

块号	页号	访问位	指针
0			
1			
2	4	0	
3			
4	2	1	
5			
6	5	0	
7	1	1	

替换指针

5 设备管理

5.1 概述

外部设备分为两大类：一类是存储型设备（如磁带机、磁盘机等），以存储大量信息和快速检索为目标，在系统中存储持久性信息，它作为内存的扩充，所以又称外存；另一类是I/O型设备（如显示器、打印机、通信设备等），它们把外界信息输入计算机，把计算结果从计算机中输出，完成计算机之间的通信或人机交互。

设备管理是操作系统中最为庞杂和琐碎的部分，通常使用I/O中断、缓冲区管理、通道、设备驱动调度等多种技术，这些措施较好地克服了由于设备和CPU速度不匹配所引起的问题，使主机和设备能够并行工作，提高设备使用效率。另一方面，操作系统将所有设备都抽象为文件，将其统一在文件系统之下，赋予文件属性，对设备的操作就类似于对文件的操作，其优点是：尽可能统一文件和设备的I/O处理；尽可能把设备文件和普通文件纳入同一保护机制下。为了方便用户或高层软件使用，设备管理还对各种设备进行抽象，配置设备驱动程序，屏蔽物理细节和操作过程，提供统一使用界面。

设备管理的基本功能有：隐蔽物理设备的细节、保证操作系统与设备无关、提高处理机和I/O设备的利用率、控制I/O设备、确保对设备的正确共享、处理错误、设备中断处理、缓冲区管理、设备分配和去配、设备驱动调度、实现虚拟设备。

5.2 I/O设备的分类

①按设备的从属关系分类：

系统设备、用户设备

②按信息交换的单位分类：

字符设备（例如键盘、打印机、显示器）、块设备（例如磁盘、磁带）。I/O型设备通常是字符设备，其与内存进行信息交换的单位是字节，一次交换一个或多个字节；存储型设备通常是块设备：又可分为顺序存取存储设备和直接存取存储设备，前者严格依赖信息的物理位置进行定位和读写（例如磁带）、后者与之相反（例如磁盘）。块指的是存储介质上连续信息组成的区域，块设备每次与内存交换一块或者多块信息。

③按使用方式/共享属性分类：

独享/独占设备（一旦被分配使用，在没被释放前，其他实体不得使用，包括所有的字符型设备和磁带机，多数低速设备属于该类，打印机也是一个典型的例子）、共享设备（在一段时间允许多个用户进程同时访问的设备，包括除磁带机以外的所有块设备，磁盘就是

一个典型的例子)、虚拟设备(将一台独享设备模拟为共享设备,具有了新特性)。

④按传输速率分类:

低速设备(传输速率为每秒钟几个字节至数百个字节的设备,例如键盘、鼠标、语音输入/输出)、中速设备(传输速率为每秒钟数千个字节至数万个字节的设备,例如打印机)、高速设备(传输速率为每秒钟数千个字节至数十兆字节的设备,例如磁盘机、磁带机、光盘机)。

5.3 设备控制器

5.1.2 设备控制器

- 传统的设备 = 机械部分+电子部分
- 电子部分在系统的控制下驱动机械部分运转,完成 I/O 操作
- 由于设备中电子部分比机械部分的速度快得多,为了降低硬件成本,将电子部分从设备中分离出来作为一个独立的部件,这就是控制器。在小型和微型机中,它常采用印刷电路卡插入计算机中。
- 分离之后的设备仅由机械部分构成,一个控制器可与多个设备相连(因为控制器的速度快于机械部分),交替地或分时地控制与其相连的设备。例如,磁盘控制器可以控制多个磁盘驱动器。

1. 为了实现设备与 CPU 的交互,设备控制器与 CPU 链接的一侧通常含有 ()

A. 控制寄存器、状态寄存器 B. I/O 地址寄存器、状态控制器

C. 中断寄存器、命令寄存器 D. 数据寄存器、控制寄存器和状态寄存器

2. 描述关于通道、设备控制器和设备之间的关系

设备控制器是处于 CPU 与 I/O 设备之间的接口,接收 CPU 发来的命令,并控制 I/O 设备工作,是一个可编址的设备。它的功能是,接收和识别命令、实现数据交换、了解设备状态以及识别设备地址。

5.4 I/O 控制方式

轮询、中断、DMA、通道

它们之间的区别在于 CPU 和设备并行工作的方式和程度不同。CPU 和设备并行工作能够提高计算机的系统效率和资源利用率。轮询方式中,CPU 全程参与 I/O 操作;中断方式中,减少 CPU 参与 I/O 操作的工作量;直至 DMA 与通道方式的出现,可以在没有 CPU 参与的情况下完成 I/O 操作,把 CPU 从 I/O 中彻底解放出来,消除了系统的性能瓶颈。

①轮询方式

又称程序直接控制方式,采用程序查询的方式直接参与数据的输入/输出,效率低下。

②中断控制方式

需要数据的进程由 CPU 发出指令启动 I/O 设备输入数据,随后该进程放弃处理机,等待输入完成,当输入完成后,I/O 控制器向 CPU 发出中断请求,CPU 收到后,转向中断服务程序,中断服务程序将输入寄存器中的数据送指定内存单元,并将原进程唤醒,继续执行,从此以后,若该进程再被调度,则从内存单元取出数据进行处理。

虽然提高了 CPU 的资源利用率,但是在响应中断请求后必须停止现行程序,转入中断处理程序再参与数据传输操作。假如设备能直接与内存进行数据交换而不占用 CPU,CPU 的资源利用率还可进一步提高。

③DMA(直接内存访问)方式

需要数据的进程由 CPU 发出指令,向 DMA 控制器写入数据存放的内存始址、传送的

字节数，并置中断位和启动位，启动 I/O 设备输入数据并允许中断，随后该进程放弃处理机等待输入完成，处理机被其它进程占据，DMA 控制器窃取 CPU 周期，将一批数据写入内存中，DMA 控制器传送完数据后向 CPU 发中断请求，CPU 响应后转向中断服务程序唤醒进程，并返回被中断程序，从此以后，该进程再被调度，从内存单元取出数据进行处理。

需要的部件有：数据移位寄存器、内存地址寄存器、字计数器、数据缓冲寄存器、设备地址寄存器、中断机制和控制逻辑。

但是一台 DMA 控制器只能控制一台设备。

④通道控制方式

通道又称 I/O 处理器，它是一个独立于 CPU 的专管输入 / 输出控制的处理器，它控制设备与内存直接进行数据交换，通道有自己的一套简单的指令系统，称为通道指令。

通道的类型有字节多路通道、数组选择通道、数组多路通道。字节多路通道按照字节交叉方式工作，数组选择通道按成组方式工作，数组多路通道仍是按数组方式工作。

需要数据的进程向 CPU 发出指令，CPU 发出启动指令指明 I/O 操作、设备号和对应的通道，该进程放弃 CPU 等待输入完成，CPU 被其它进程占据，通道接收到 CPU 发来的启动指令后，取出内存中的通道程序执行，控制设备将数据传送到内存指定区域，传送完数据后，通道向 CPU 发中断请求，CPU 响应后转向中断服务程序，唤醒进程，并返回被中断程序，从此以后该进程再被调度，从内存取出数据进行处理。

一个通道可以控制多个设备。

上面 4 种方式的工作过程总结以及区别和联系：

轮询、中断、DMA、通道之间的区别在于 CPU 和设备并行工作的方式和程度不同，CPU 和设备并行工作能够提高计算机的系统效率和资源利用率。轮询方式中，CPU 全程参与 I/O 操作；中断方式中，减少 CPU 参与 I/O 操作的工作量；直至 DMA 与通道方式的出现，可以在没有 CPU 参与的情况下完成数据传输。这四种方式，依次把 CPU 逐渐解放出来，参与地越来越少，化解了系统的性能瓶颈。

轮询的优点是控制简单，不需要很多硬件支持，缺点是 CPU 和外设只能串行工作，CPU 大部分时间处于循环测试状态，利用率低；中断的优点是能实现 CPU 与设备、设备与设备的并行操作，CPU 利用率提高，但是一次数据传送中断次数较多，耗去大量 CPU 时间；DMA 的优点是一批数据传送完成后才中断 CPU，减少 CPU 中断处理次数，数据传输过程无需 CPU 干预，但某些操作仍由 CPU 控制，多个 DMA 控制器使用不经济；通道的优点是 CPU 仅在 I/O 操作开始和结束时处理相关事宜，其余时间均与通道并行工作，一个通道可控制多台设备，但是价格较高。

5.5 I/O 软件原理

系统调用是操作系统内核与应用程序之间的接口，设备驱动程序是操作系统内核与机器硬件之间的接口。设备驱动程序为应用程序屏蔽了硬件的细节，这样在应用程序看来，硬件设备只是一个设备文件，应用程序可以和操作普通文件一样对硬件设备进行操作。设备驱动程序是内核的一部分。

I/O 软件设计目标和原则

I/O 软件的总体设计目标是高效率 and 通用性。在改善设备效率的过程中,最应关注的是磁盘 I/O 操作的效率。通用性意味着用统一标准的方法来管理所有设备。为了达到这些目标,把软件组织成层次结构,低层软件用来屏蔽硬件细节,高层软件向用户提供简洁、友善的界面。I/O 软件设计需要考虑的问题有:

- (1) 设备无关性。程序员编写访问文件数据的程序时,与具体物理设备无关。
- (2) 出错处理。数据传输过程中产生的错误应该在尽可能靠近硬件的地方处理,低层软件能够解决的错误不让高层软件感知。
- (3) 同步/异步传输。CPU 在启动 I/O 操作后既可继续执行其他工作,直至中断到达,称此为异步传输;又可采用阻塞方式,让启动 I/O 操作的进程挂起等待,直至数据传输完成,称此为同步传输,I/O 软件应支持这两种工作方式。
- (4) 缓冲技术。建立数据缓冲区,让数据的到达率与离去率相匹配,以提高系统吞吐率。

为了合理、高效地解决以上问题,操作系统把 I/O 软件依次组织成 4 个层次:I/O 中断处理程序;I/O 设备驱动程序;独立于设备的 I/O 软件 and 用户空间的 I/O 软件。

SPOOLing 软件就是用户空间的 I/O 软件,并非所有的用户层 I/O 软件都由库函数构成,SPOOLing 就是在内核外运行的系统 I/O 软件,采用预输入、缓输出、井管理技术,是多道程序设计系统中处理独占型设备的一种方法,创建守护进程和特殊目录解决独占性设备的空占问题。

5.6 缓冲管理

缓冲技术是为了解决 CPU 与设备之间速度不匹配的矛盾,协调逻辑记录大小与物理记录的大小不一致的问题,提高 CPU 和设备的并行性,减少 I/O 操作对 CPU 的中断次数,放宽对 CPU 中断响应时间的要求。

缓冲的两种实现方法:

一种是采用专门硬件寄存器,称为硬件缓冲;一种是在存储器里开辟一个区域,专门用来存放临时输入/输出数据,称为软件缓冲。

缓冲技术的分类:

单缓冲(串行)、双缓冲、多缓冲、循环缓冲、缓冲池等。

5.7 设备分配

①设备独立性(设备无关性)

为提高 OS 的可适应性和可扩展性,而将应用程序独立于具体使用的物理设备,即用户不指定物理设备,而指定逻辑设备使得用户作业和物理设备分离开,通过其他途径建立逻辑设备和物理设备的映射,这种特性叫设备独立性。

好处:

应用程序与具体的物理设备无关,系统增减或变更设备时对源程序不必加以任何修改,设备分配灵活、易于重定向、容错能力强,一个发生故障可以用另一个替换,提高物理设备的共享性和系统的可靠性,能更有效地利用设备资源,实现多道程序设计。

注:

逻辑设备名到物理设备名的映射是通过逻辑设备表 LUT来实现的。

②设备分配方式

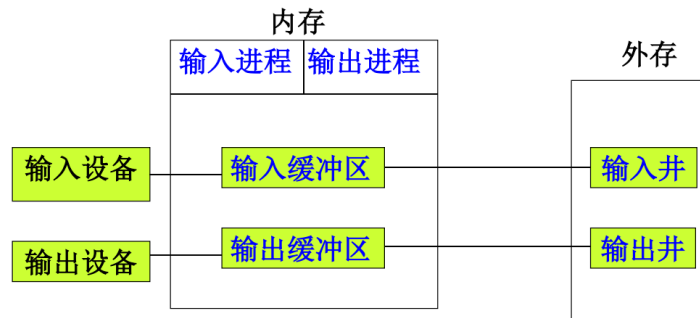
从设备的物理特性来看,设备可以分为独占型设备、共享设备、虚拟设备三类,相应的

管理和分配技术分为静态分配、动态分配、虚拟分配。

5.7 虚拟设备（SPOOLing 技术）

虚拟设备是指通过虚拟技术将一台独占设备变换为若干台逻辑设备，供若干个用户进程使用，这种经过虚拟技术处理后的设备就是虚拟设备。

在操作系统中，引入虚拟设备的原因是为克服独占设备速度较慢、降低设备资源利用率的缺点，从而提高资源利用率。n



SPOOLing 技术是在主机控制下基于联机操作的脱机工作方式，为了缓和 CPU 的高速性与 I/O 设备的低速性间的矛盾而引入，该技术在外围控制机的控制下实现低速 I/O 设备与高速磁盘之间的数据传送：采用预输入、缓输出、井管理技术，输入井和输出井是磁盘上开辟出的存储空间，用于模拟输入和输出设备，输出缓冲区和输入缓冲区在内存中，在多道程序下，用程序来模拟外围控制机，实现将数据从磁盘传送到低速的输出设备上，从而可在主机的直接控制下，实现脱机输入、输出功能，进而实现外围操作与 CPU 对数据处理的并行操作，这种在联机情况下实现的同时外围操作是对脱机输入、输出工作的模拟，是操作系统中采用的一项将独占设备改造成为共享设备的技术。

这种技术提高了 I/O 速度（对低速设备的操作变为对输入/输出井的操作），将独占设备改造为共享设备，实现了虚拟设备的功能。虚拟设备实现了同步向异步的转变，提高了进程的并发度和执行效率，以及设备的利用率，但其付出的代价是空间的开销，是操作系统用空间换时间的典型范例。

5.8 磁盘存储器

1.[简答题]某磁盘系统中，磁头访问143号磁道后，正在访问125号磁道。正在等待的访问请求柱面序列为：147, 91, 177, 94, 150, 102, 175, 130。已知磁头移动一个磁道需要0.1ms。分别采用下列磁盘调度算法时，磁道的访问顺序是什么，总的寻道时间是多少ms？

SSTF(1) 最短寻道时间优先 (Shortest seek time first)

(2) 电梯调度算法

1) 125 → 130 → 147 → 150 → 175 → 177 → 102 → 94 → 91

长度：5 + 17 + 3 + 25 + 2 + 75 + 8 + 3 = 138 (个)

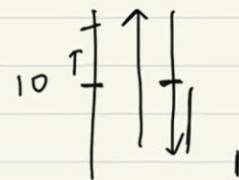
$t_1: 138 \times 0.1ms = 13.8ms$

2) 125 → 102 → 94 → 91 → 130 → 150 → 175 → 177

长度：23 + 8 + 3 + 39 + 20 + 25 + 2 = 120 (个)

$t_2: 120 \times 0.1ms = 12ms$

143 → 125 N



操作系统 磁盘调度算法

1. 先来先服务 FCFS

优点: 公平, 简单, 不会出现有请求长期得不到满足的情况

缺点: 平均寻道时间长

2. 最短寻道时间优先 SSTF

下一次访问的磁道与当前所在磁道最近

优点: 每次的寻道时间最短

缺点: 不能保证平均寻道时间最短, 优先级低的进程可能“饥饿”。

3. 扫描算法 SCAN

既考虑下次与当前所在的距离, 又考虑磁头当前的移动方向

优点: 防止“饥饿”

4. 循环扫描算法 CSCAN

优点: 寻道性能好, 防止“饥饿”

5.9 磁盘高速缓存

指内存中的一部分存储空间, 用来暂存从磁盘读出的一系列盘块中的信息, 所以它是一组在逻辑上属于磁盘, 而物理上是驻留在内存中的盘块。由于程序局部性原理, 它的使用可以减少磁盘与内存之间的 I/O 传送次数。

6 文件管理

所有的计算机应用程序都需要存储和检索信息。进程运行时, 可在它自己的地址空间存储一定量的信息, 但存储容量受虚拟地址空间大小的限制。在进程的地址空间保存信息的另一个问题是, 当进程终止时, 它保存的信息也随之丢失。还有一个问题是, 经常需要多个进程同时存取相同信息, 那么就要使信息本身独立于任一进程。因此, 长期存储信息有三个基本要求: (1) 能够存储大量信息 (2) 使用信息的进程终止时, 信息仍旧存在 (3) 必须能使多个进程并发存取有关信息。

解决所有这些问题的通常做法是: 把信息以一种单元的形式, 也就是所谓的文件, 存储在磁盘或其他外部介质上。进程在需要时可以读取这些信息或写入新的信息。存储在文件中的信息必须是永久性的, 也即信息不会因为创建和终止进程而受到影响。只有在文件的所有者显式地删除文件时, 文件才会消失。

6.1 文件

数据项分为基本数据项（最小的逻辑数据单位）和组合数据项，而记录正是一组相关数据项的集合。

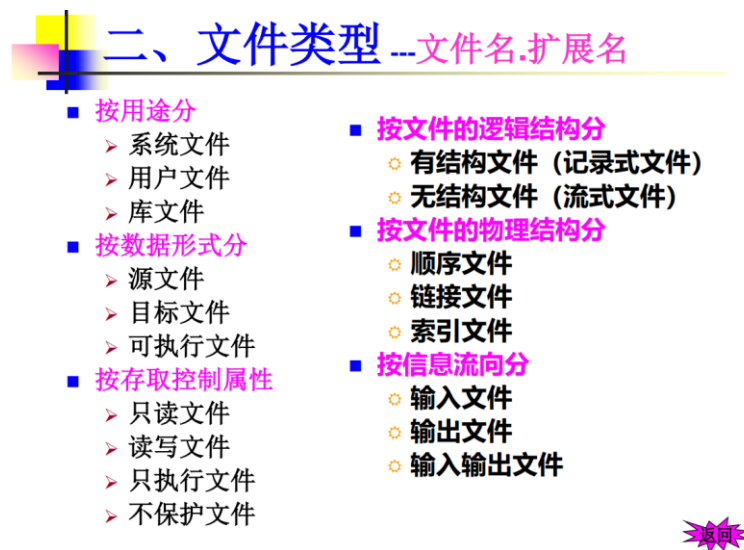
文件是指记录在外存上的具有文件名的一组相关信息的集合。可分为有结构文件和无结构文件两种。有结构文件是由若干个相关记录组成，而无结构文件则被看成一个字符流。

6.2 文件命名

个字符,现在文件名最长可达 255 个字符。扩展名用于定义和区分文件类型,说明文件的内容和内部格式。系统有一些约定的扩展名,例如,.txt 指明纯文本文件,.exe 表示可执行二进制代码文件,.obj 表示编译或汇编生成的目标文件。定义文件扩展名是一种习惯,如汇编语言源程序的扩展名取.asm,C 语言编译器要求被编译的源程序的扩展名取.c。

一般来说,文件名由文件名和扩展名两部分组成。

6.3 文件类型



6.4 文件组织方式

对任一文件存在着两种形式的结构：文件的逻辑结构（文件组织）对应用户如何使用文件。这是从用户观点出发,所观察到的文件组织形式,是用户可以直接处理的数据及其结构,它独立于物理特性。文件的物理结构（文件的存储结构）是指文件在外存上的存储组织形式,与存储介质的存储性能有关,分为顺序、链接及索引。

①文件的逻辑结构

类型：流式文件、记录文件（顺序、索引、索引顺序）

流式文件是无结构的,由字符流构成;记录式文件是有结构的,由一个以上的记录构成。

②文件的物理结构（外存分配方式）

类型：顺序（连续）分配、链接分配（用到文件分配表 FAT）、索引分配（直接、多级、混合）

文件的存储单位是簇,文件的存储空间通常由多个分立的簇组成,每个簇包含若干个连续的扇区或者块。

6.5 文件存取方法

顺序存取、直接存取、索引存取

6.6 目录管理

文件目录会给整个访问带来什么样的好处？

实现**按名存取**、提高对目录的检索速度、文件共享、允许文件重名。

①文件控制块 FCB（亦称目录项）

从文件管理的角度看，文件由 FCB 和文件体两部分组成。

文件控制块是操作系统为管理文件而设置的数据结构，存放文件有关的说明信息，是文件存在的标志。每个文件都有一个 FCB，它是作为**目录项**存储于**目录文件**（由文件目录构成的用来维护文件系统结构的系统文件）中的，被保存在**外存空间**，当要访问一个文件的时候，应当能够根据文件名字找到它所对应的 FCB。

文件目录（FCB 的集合）：

用于检索文件的目录，是由目录项构成的有序序列。好处：实现按名存取、提高对目录的检索速度、文件共享、允许文件重名。

目录项：

构成文件目录的项目，目录项就是 FCB。

目录文件：

为了实现对文件目录的管理，将文件目录以文件的形式保存在外存，这个文件就是目录文件。

文件目录和目录文件是同一事物的两种称谓。从用途角度来看称其为文件目录，从实现角度来看称其为目录文件。目录文件不会为空，至少包含.和..

②文件索引节点 inode

FCB 中的文件名和其他管理信息分开，其他信息单独组成一个数据结构，称为索引节点 **inode**，索引节点的位置由 **inode** 号标识。这样一来，目录项中只剩下文件名和 **inode** 号，此为基本目录项。引入索引节点是因为文件控制块的集合组成目录，“按名存取”时需根据文件名搜索目录，此时需将文件控制块的所有信息读入内存，导致目录检索速度慢。为提高检索速度，将文件控制块中除文件名以外的其他信息分离出来放入索引结点，目录中的目录项变为文件名和对应的索引结点数，从而压缩目录，提高目录检索速度，这个就是它的好处。

③单级目录结构

整个系统中只有一张目录表。易实现按名存取，但易重名，查找速度慢，不便于共享。

④两级目录结构

系统中为所有用户建立一个主文件目录 MFD，为每个用户建立一个单独的用户文件目录 UFD。每个文件均由系统中的用户目录名和用户目录中的文件名两部分进行标识，不会重名，但是也不便于共享。

⑤树型目录结构

允许用户在两级目录中建立自己的子目录，形成多级目录结构。优点是层次结构清晰，实现分组，便于管理和保护；解决重名问题；每个目录下的文件数目较少，查找速度加快。缺点是查找一个文件会按路径名逐层检查，由于每个文件都放在外存，多次访盘影响速度。

6.7 目录查询技术

线性检索法（顺序检索法）、Hash 方法

6.8 文件存储空间的管理

空闲表法和空闲链法、位示图法、成组链接法

6.9 文件共享

基于索引节点的共享方式是**硬链接**，利用符号链实现的共享是**软链接**。

硬链接和软链接是操作系统文件系统中两种重要的链接机制，核心区别在于是否共享 **inode** 索引节点，硬链接是原文件的别名，与原文件共用一个 **inode**，软链接是独立文件，仅存储原文件的路径指针，指向原文件的 **inode**。

硬链接的方式，任何用户对文件进行附加操作或修改，会引起的相应索引结点内容的改变，对其他用户均可见，实现共享。优点是原文件的删除或移动不影响访问，访问速度更快，没有悬空链接，数据安全性更高，权限与原文件一致且同步更新，占用磁盘空间少。缺点是不能跨文件系统创建，不能链接目录，只能对已存在的文件创建，无法单独修改权限或属性，仅记录 inode，删除原文件后无法恢复原文件的路径和名称。

软链接的方式，优点是支持跨文件系统创建，可以链接目录，可以对不存在的文件创建，可以单独设置权限和属性，可以链接到不同类型的文件，删除软链接不影响原文件。缺点是根据给定的文件路径名去查找目录，访问文件的开销大，启动磁盘的频率高，每一个共享文件有几个文件名，会增加遍历共享文件的次数。

6.10 文件保护

几种方式：口令保护、加密保护、访问控制（保护域和访问矩阵）。

